

AUDITORIA FORENSE — ETAPA 1: MAPEAMENTO COMPLETO DA ARQUITETURA DE GERAÇÃO DE VÍDEO

Projeto: Fluxyra SaaS — `/home/ubuntu/github_repos/saas`

Branch: `feat/f3-piapi-landing-pricing` · **Último commit:** `a93267b`

Data da auditoria: 26/07/2026

Escopo: somente leitura/análise de código. Nenhum arquivo de código foi alterado; nenhuma API externa foi chamada; nenhum vídeo foi gerado.

1. DIAGRAMA DO FLUXO COMPLETO

1.1 Fluxo principal (clique em Generate → vídeo no feed)

FRONTEND (browser)

```

src/app/(dashboard)/studio/page.tsx
├── <StudioFeed/> (feed + polling)
├── <GenerationDock/> (UI de geração)

GenerationDock (src/components/studio/generation-dock.tsx)
├── estado global: useStudioStore (Zustand)
├── estado local : models, credits, loading, multi-shot, refs de <input file>
├── carrega modelos ..... GET /api/models?type=video (linha 676)
├── carrega créditos ..... GET /api/me (linha 686)
├── upload de referências .. POST /api/upload → bucket "uploads" (linha 236)
├── "Wise enhance" ..... POST /api/assist (LLM reescreve o prompt, l.924)
├── CLIQUE EM "Generate" → handleGenerate() (linha 1047)
│   ├── valida: prompt não vazio
│   ├── valida: selectedModel.available === true
│   ├── valida: credits >= totalCost (insufficient, linha 808)
│   ├── addOptimistic() → card otimista imediato no feed
│   └── submitSingleGeneration() (linha 953)
│       ├── monta body: { prompt, model_uuid, aspect_ratio, quality, duration,
│       │               resolution, negative_prompt, start_image_url, end_image_url,
│       │               reference_images[], reference_videos[], reference_audios[],
│       │               shots[] (multi-shot), with_audio }
│       └── → POST /api/generate/video

```

BACKEND (Next.js API Routes)

```

POST /api/generate/video (src/app/api/generate/video/route.ts)

1. auth Supabase (cookie) ..... linhas 9–16
2. valida prompt + model_uuid ..... linhas 59–64
3. busca ai_models (id=model_uuid, is_active) ..... linhas 67–77
4. verifica créditos (profiles.credits_balance) ... linhas 94–105 → 402
5. gating de plano (planAllows, min_plan) ..... linhas 107–113 → 403
6. DÉBITO de créditos (update profiles) ..... linhas 116–120
7. INSERT generations (status="pending",
   params inclui output_key do modelo) ..... linhas 128–153
8. INSERT credit_transactions (amount negativo,
   reason="generation") ..... linhas 161–166
9. buildVideoPayload() (lib/piapi/client.ts) ..... linhas 172–186
10. submitVideoTask() → POST PiAPI /api/v1/task .... linha 190
11. sucesso: UPDATE generations
    {provider_task_id, status="processing"} ..... linhas 200–203
11b. ERRO PiAPI: ESTORNO (restaura credits_balance,

```

```
generations.status="failed",
credit_transactions reason="refund") ..... linhas 211–235
→ responde 402 (saldo PiAPI) ou 502
```

```

      | task_id
      ▼
    PiAPI (provider externo)
    POST https://api.piapi.ai/api/v1/task
    GET  https://api.piapi.ai/api/v1/task/{id}

```

POLLING DE STATUS

StudioFeed (src/components/studio/studio-feed.tsx)

setInterval a cada POLL_MS = 3000 ms (linhas 26, 87–106)

para cada geração pending/processing:

GET /api/generate/status?id=<generation_id>

GET /api/generate/status (src/app/api/generate/status/route.ts)

1. auth + busca generations (dono) linhas 14–37
2. se completed/failed no banco → retorna direto .. linhas 40–46
3. getTaskStatus(provider_task_id) → PiAPI linha 56
4. state === "completed":
 - a. extractVideoUrl(output, output_key) linhas 76–79
 - b. fetch(providerUrl) → download do MP4 linhas 101–102
 - c. upload → Supabase Storage bucket "assets"

path: {user_id}/video/{generation_id}.mp4 . linhas 99–115
 - d. getPublicUrl → finalUrl linhas 120–123
 - e. UPDATE generations {status="completed",

result_url=finalUrl} (service role) linhas 129–136
5. state === "failed":
 - a. lê error.message + logs[] da PiAPI linhas 141–165
 - b. traduz causa real p/ PT-BR (real person,

plan limit, nsfw/content policy) linhas 165–185
 - c. UPDATE generations {status="failed",

```

    error_message} ..... linhas 187-194

d. ESTORNO automático de créditos +

    credit_transactions reason="refund" ..... linhas 196-215

```

```

    | status muda → changed=true
    ▼
StudioFeed → fetchGenerations() → GET /api/generations?limit=80
    (src/app/api/generations/route.ts – SELECT generations + JOIN ai_models)
    → renderiza <video src={result_url}> ou <FailedCard error_message/>

```

1.2 Fluxo de créditos (resumo)

Exibição: GenerationDock → GET /api/me → profiles.credits_balance
 Sidebar → useCredits() → GET /api/credits → profiles.credits_balance

Verificação (frontend): insufficient = credits < totalCost (dock, linha 808) → bloqueia botão/toast

Verificação (backend): profiles.credits_balance < ai_models.credit_cost → HTTP 402 (video/route.ts l.100)

Débito: UPDATE profiles.credits_balance (video/route.ts l.116-120)
 + INSERT credit_transactions {amount: -credit_cost, reason: "generation"} (l.161-166)

Estorno 1: falha imediata na PiAPI (submit) → video/route.ts l.217-224 (reason: "refund")

Estorno 2: falha assíncrona (task failed) → status/route.ts l.196-215 (reason: "refund")

Estorno 3: via webhook PiAPI (se acionado) → webhooks/piapi/route.ts l.49-71 (reason: "refund")

Recarga UI: loadCredits() re-chamado após gerar (dock l.1102); useCredits refetch a cada 30 s

2. LISTA COMPLETA DE ARQUIVOS ENVOLVIDOS (por camada)

2.1 Página / Componentes React

Arquivo	Responsabilidade	Principais funções/exports
<code>src/app/(dashboard)/studio/page.tsx</code>	Página do Studio; monta o feed e o dock	default export <code>StudioPage</code> → <code><StudioFeed/></code> + <code><GenerationDock/></code>
<code>src/components/studio/generation-dock.tsx</code> (2 226 linhas)	UI central de geração: tarefa de prompt, seletor de modelos agrupado por família, aspect ratio, resolution, duration (slider ou enum por backend), quality (low/medium/high), toggle de áudio (<code>audioEnabled</code>), abas de referência Start/End Frame vs Omni Reference, upload de imagens (máx. 9) / vídeos (máx. 3) / áudios (máx. 3), Multi-Shot (storyboard Kling 3.0, auto/custom até 6 shots), painel Assist (presets), botão “Wise enhance”, negative prompt, batch (só imagem), botão Generate	<code>GenerationDock</code> (l.591); internas: <code>handleGenerate</code> (l.1047), <code>submitSingleGeneration</code> (l.953), <code>handleWiseEnhance</code> (l.924), <code>uploadReferenceFile</code> (l.233), <code>loadModels</code> (l.674), <code>loadCredits</code> (l.685), <code>loadAssets</code> (l.697); derivados: <code>aspectOptions</code> (l.768), <code>resolutionOptions</code> (l.786), <code>durationSnapValues</code> (l.814), <code>totalCost</code> / <code>insufficient</code> (l.807–808)
<code>src/components/studio/studio-feed.tsx</code> (515 linhas)	Feed em grid; cards otimizadas/pending/failed; polling de status a cada 3 s ; limpeza de falhas	<code>StudioFeed</code> ; <code>POLL_MS = 3000</code> (l.26); <code>polling</code> (l.87–106); <code>fetchGenerations</code> (l.54); <code>clearAllFailed</code> (l.169); <code>FailedCard</code> , <code>PendingCard</code> , <code>GenerationCard</code>
<code>src/components/studio/media-lightbox.tsx</code> (480 linhas)	Modal de detalhe da mídia; ações “Use as Reference” / “Create Video” (alimenta o store), favoritar (PATCH) e excluir (DELETE)	<code>MediaLightbox</code> ; <code>handleUseAsReference</code> , <code>handleCreateVideo</code> , <code>handleFavorite</code> , <code>handleDelete</code>
<code>src/components/studio/media-gallery.tsx</code>	Não é usado em nenhuma página (nenhum import fora do próprio arquivo) — componente órfão	<code>MediaGallery</code>
<code>src/components/shared/sidebar.tsx</code>	Navegação + badge de créditos (usa <code>useCredits</code>)	<code>Sidebar</code>
<code>src/components/shared/topbar.tsx</code>		<code>Topbar</code>

Arquivo	Responsabilidade	Principais funções/exports
	Filtro de visualização (Tudo/ Imagem/Vídeo/Áudio) via <code>viewFilter</code> do store	

2.2 Hooks

Arquivo	Responsabilidade	Observação
<code>src/hooks/use-credits.ts</code>	React Query: <code>GET /api/credits</code> , <code>refetch 30 s</code>	usado por <code>Sidebar</code> . O <code>GenerationDock</code> não usa este hook — faz <code>fetch("/api/me")</code> próprio (duplicação, ver riscos)
<code>src/hooks/use-generation.ts</code>	Hook de geração com polling próprio (<code>GENERATION_POLL_INTERVAL_MS</code>)	CÓDIGO MORTO: <code>grep</code> não encontra nenhum consumidor — <code>GenerationDock</code> submete via <code>fetch</code> direto e o polling real é do <code>StudioFeed</code>

2.3 Store (Zustand)

Arquivo	Responsabilidade	Estado relevante para vídeo
<code>src/stores/use-studio-store.ts</code> (206 linhas)	Estado global do Studio	<code>activeTab</code> , <code>prompt</code> , <code>negativePrompt</code> , <code>selectedModelId</code> , <code>aspectRatio</code> (default <code>"1:1"</code>), <code>duration</code> (default <code>4</code>), <code>resolution</code> (default <code>"1080p"</code>), <code>quality</code> (default <code>"high"</code>), <code>referenceTab</code> (<code>"start-end"</code> <code>"omni"</code>), <code>startImageUrl</code> , <code>endImageUrl</code> , <code>referenceImageUrl</code> , <code>referenceImages[]</code> (máx 9), <code>referenceVideos[]</code> (máx 3), <code>referenceAudios[]</code> (máx 3), <code>viewFilter</code> , <code>refreshKey</code> / <code>triggerRefresh</code> , <code>cards otimizadas</code> (<code>addOptimistic</code> / <code>clearOptimistic</code>), <code>resetParams</code>

2.4 Services / Libs

Arquivo	Responsabilidade	Principais exports
<code>src/lib/piapi/client.ts</code> (691 linhas)	Cliente PiAPI + adapter de payloads por backend (detalhe na seção 2.6)	<code>buildVideoPayload</code> (l.288), <code>submitVideoTask</code> (l.549), <code>getTaskStatus</code> (l.654), <code>extractVideoUrl</code> (l.564), <code>extractResultUrl</code> (l.661), <code>generateImage</code> , <code>generateImageGptSync</code> , <code>generateAudio</code> , interfaces <code>PiAPITaskResponse</code> , <code>PiAPIStatusResponse</code> (com <code>logs?: string[]</code> , l.57), <code>VideoModelParams</code> (l.247), <code>BuildVideoArgs</code> (l.260), classe <code>PiAPIError</code> (l.62), <code>piapiFetch</code> (l.73, injeta <code>x-api-key</code> de <code>PIAPI_API_KEY</code>)
<code>src/lib/plans.ts</code>	Gating de plano (<code>free < starter < pro < agency</code>)	<code>planAllows(userPlan, minPlan)</code>
<code>src/lib/constants.ts</code>	Regras de negócio	<code>GENERATION_POLL_INTERVAL_MS = 3000</code> , <code>WELCOME_CREDITS = 10</code> , <code>TOPUP_PACKS</code> , <code>HIGH_COST_COOLDOWN_SECONDS = 30</code> (constante definida, não encontrei aplicação do cooldown no fluxo de vídeo)
<code>src/lib/assist-presets.ts</code> (136 linhas)	Categorias/presets do painel Assist (texto inserido no prompt)	<code>ASSIST_CATEGORIES</code> , <code>Assist-Category</code>
<code>src/lib/supabase/server.ts</code>	Cliente Supabase SSR com cookie do usuário (anon key, respeita RLS)	<code>createClient()</code>
<code>src/lib/supabase/service.ts</code>	Cliente com <code>SUPA-BASE_SERVICE_ROLE_KEY</code> (ignora RLS) — usado p/ Storage e updates confiáveis	<code>createServiceClient()</code>
<code>src/lib/supabase/client.ts</code> , <code>middleware.ts</code> , <code>queries.ts</code>	Cliente browser, middleware de sessão, queries auxiliares	—
		—

Arquivo	Responsabilidade	Principais exports
src/lib/atlas/client.ts , src/lib/abacus/client.ts , src/lib/fal/	Providers de áudio/imagem — não participam do fluxo de vídeo	

2.5 Rotas de API (`src/app/api/**`)

Rota	Método	Responsabilidade no fluxo de vídeo
<code>generate/video/route.ts</code> (241 l.)	POST	Rota principal: auth → modelo → créditos → plano → débito → INSERT generations → <code>buildVideoPayload</code> → PiAPI → <code>provider_task_id</code> . Estorno em falha de submit
<code>generate/status/route.ts</code> (229 l.)	GET	Polling: consulta PiAPI, extrai URL (<code>output_key</code>), baixa MP4, sobe no bucket <code>assets</code> , persiste <code>result_url</code> ; em falha parseia <code>logs[]</code> p/ mensagem PT-BR e estorna créditos
<code>models/route.ts</code> (86 l.)	GET	Catálogo <code>ai_models</code> ativo; calcula <code>available</code> via lista fixa <code>WORKING_BACKENDS</code> (1.7–25); expõe <code>backend</code> , <code>task_type</code> , <code>dur_min/max</code> , <code>credit_cost</code> , <code>min_plan</code>
<code>me/route.ts</code>	GET	{ <code>credits</code> , <code>plan</code> } do <code>profiles</code> — usado pelo dock
<code>credits/route.ts</code>	GET	Idêntico em essência ao <code>/api/me</code> — usado pelo <code>useCredits</code> (Sidebar)
<code>generations/route.ts</code>	GET	Lista gerações do usuário (JOIN <code>ai_models</code> p/ nome do modelo) — alimenta o feed
<code>generations/[id]/route.ts</code>	DELETE / PATCH	Excluir geração (cards de falha) / favoritar
<code>upload/route.ts</code> (94 l.)	POST	Upload multipart (máx. 20 MB, mime image/video/audio) → bucket <code>uploads</code> , path <code>{user_id}/references/{kind}/{uuid}.{ext}</code> → URL pública usada como referência
<code>assist/route.ts</code> (82 l.)	POST	“Wise enhance”: reescreve o prompt via LLM (endpoint

Rota	Método	Responsabilidade no fluxo de vídeo
		OpenAI-compatible, <code>ABA-CUS_API_KEY / LLM_BASE_URL</code>)
<code>assets/route.ts</code>	GET	Lista <code>assets</code> do usuário para o painel “@” do dock
<code>webhooks/piapi/route.ts</code> (148 l.)	POST	Webhook de conclusão da PiAPI (ver seção 2.9 — existe mas não é registrado nas tasks)
<code>generate/image/route.ts</code> , <code>generate/audio/route.ts</code>	POST	Fluxos de imagem/áudio (fora do escopo de vídeo, compartilham o padrão débito/estorno)
<code>stripe/*</code> , <code>webhooks/stripe</code> , <code>voices/preview</code>	—	Billing/TTS — não participam da geração de vídeo

2.6 Adapter de payloads por backend (`buildVideoPayload` , `src/lib/piapi/client.ts` l.288-544)

O roteamento é decidido por `ai_models.params.backend` (campo `VideoModelParams.backend`):

Backend	Linhas	model/task_type enviado à PiAPI	Particularidades
seedance	309–359	model=seedance , task_type=seedance-2 [-fast\ -mini]	resolução por quality (480p/720p/1080p; fast/mini limitados a 720p); image_urls (1-2 = first/last frame; até 9 = omni); video_urls , audio_urls (só com imagem/vídeo); auto_upload_assets: true quando há imagens (l.357); output em output.video_url
Wan	362–387	model=Wan , task_type=wan26-tx- t2video / wan26- img2video	duração snap [5,10,15]; resolução "720P"/"1080P" (P maiúsculo); campo de imagem = image ; aspect_ratio só em txt2video; áudio nativo default true
hailuo	390–404	model=hailuo , task_type=video_gene- ration	duração snap 6/10; 1080 só com 6 s; não aceita aspect_ratio; imagem = image_url ; hailuo_model (default v2.3)
veo3 / veo3.1	407–429	model=veo3[.1] , task_type=veo3[- fast]/veo3.1-video	duração enum [4,6,8] como string "Xs" ; aspect só 16:9/9:16; generate_audio
klimg-turbo	435–453	model=klimg + ver- sion="2.5" + mode="turbo"	workaround da auditoria 2 (model=klimg-turbo dá 500 na PiAPI); duração 5/10 por quality
klimg omni	460–492		images[] (máx 4) + prefixo @image_N no

Backend	Linhas	model/task_type enviado à PiAPI	Particularidades
		model=kling , task_type=omni_video_generation	prompt; vídeo ref = campo video + @video ; enable_audio
kling 3.0	495-525	model=kling , task_type=video_generation , version=3.0	mode std/pro por quality; duração livre 3-15; Multi-Shot (prefer_multi_shots + multi_shots[] máx 6, l.514-522)
kling classic (default)	527-543	model=kling , task_type=video_generation	versão 1.5/1.6/2.1/2.5/2.6; duração ENUM 5/10 (por quality, não pela escolha do usuário); imagem = image_url / image_tail_url

Extração do resultado: `extractVideoUrl(output, output_key)` (l.564-585) — `output.video` vs `output.video_url` conforme `ai_models.params.output_key` , gravado em `generations.params.output_key` no momento do submit (video/route.ts l.125, 147).

2.7 Banco de dados (Supabase Postgres)

Fonte: `supabase/SCHEMA_PRODUCAO.md` (schema real em produção) — **atenção:** `supabase/migrations/0001_init.sql` descreve um schema v1 divergente (colunas `modality` , `credits_charged` , `credits`), que NÃO é o usado pelo código (ver riscos).

Tabela	Colunas relevantes ao vídeo	Quem lê	Quem escreve
generations	id, user_id, model_id (FK ai_models), type (video), prompt, negative_prompt, params (jsonb: aspect_ratio, duration, resolution, start/end_image_url, reference_, shots, quality, output_key*), status (pending→processing→completed/failed), provider_task_id, result_url, credits_used, error_message, created_at, updated_at	GET /api/generations, GET /api/generate/status, DELETE/PATCH /api/generations/[id], webhook	POST /api/generate/video (INSERT + UPDATE), GET /api/generate/status (UPDATE via service role), webhook (UPDATE)
profiles	id, credits_balance, plan, plan_credits_monthly	/api/me, /api/credits, /api/generate/video	/api/generate/video (débito/estorno), /api/generate/status (estorno), webhooks (estorno/Stripe)
ai_models	id, name, provider, type, model_id, credit_cost, min_plan, is_active, sort_order, params (jsonb: backend, task_type, output_key, kling_version, kling_mode, hailuo_model, dur_min/max, family, badge ...)	/api/models, /api/generate/video, /api/generations (JOIN)	scripts em scripts/*.mjs (seed/manutenção manual)
credit_transactions	user_id, amount (±), reason (generation /	(auditoria/extrato)	/api/generate/video, /api/gener-

Tabela	Colunas relevantes ao vídeo	Quem lê	Quem escreve
	refund (...), related_job_id		ate/status , webhooks/piapi
assets	id , user_id , category , name , image_url	/api/assets (painel “@” do dock)	fluxos externos ao vídeo (não é escrita pela pipeline de vídeo atual)
generation_jobs	tabela legada v1 — não usada pelo fluxo atual	—	—

2.8 Storage (Supabase Storage)

Bucket	Uso	Path	Quem escreve
uploads (público)	Imagens/vídeos/áudios de referência enviados pelo usuário	{user_id}/references/{kind}/{uuid}.{ext}	POST /api/upload (service role)
assets (público)	Mídia final gerada (vídeo MP4)	{user_id}/video/{generation_id}.mp4 (status route, l.99) — obs.: o webhook usa path diferente: {user_id}/videos/{id}.mp4 (plural, webhook l.90-92)	GET /api/generate/status (service role) e POST /api/webhooks/piapi

2.9 Webhooks, polling, filas e workers

- **Polling (mecanismo real em uso):** StudioFeed roda `setInterval` de **3 000 ms** (`POLL_MS`, `studio-feed.tsx` l.26) e, para cada geração `pending / processing`, chama `GET /api/generate/status?id=...`. É o backend dessa rota que consulta a PiAPI, baixa e persiste o vídeo. Ou seja, **a finalização da geração depende do browser do usuário estar aberto** (ver riscos). Há um segundo mecanismo de polling em `src/hooks/use-generation.ts` (intervalo `GENERATION_POLL_INTERVAL_MS = 3000`), porém **não é usado por nenhum componente** (código morto).
- **Webhook:** existe a rota `POST /api/webhooks/piapi` (`route.ts`, 148 linhas) que atualiza a geração e estorna créditos. Porém **buildVideoPayload envia apenas** `config: { service_mode: "public" }` (`client.ts` l.302) — **nenhum** `webhook_config` / **URL de callback é registrado na criação da task**, e não há qualquer outra referência a webhook em `src/lib` ou `src/app/api/generate`. Conclusão: o webhook **existe como código, mas não é acionado pela PiAPI** no fluxo atual (a menos que configurado externamente no painel da PiAPI — não há evidência disso no repositório).

- **Filas e workers: não existem no projeto.** Não há BullMQ, cron, Edge Functions, `instrumentation.ts` ou qualquer processo em background. Toda a orquestração é síncrona nas rotas API + polling do frontend.
-

3. TABELA DE DEPENDÊNCIAS (quem chama quem)

#	Origem	Destino	Via
1	studio/page.tsx	StudioFeed , GenerationDock	render
2	GenerationDock	useStudioStore	leitura/escrita de todo o estado de geração
3	GenerationDock	GET /api/models? type=video	fetch (catálogo)
4	GenerationDock	GET /api/me	fetch (créditos exibidos/validação)
5	GenerationDock	POST /api/upload	fetch (refs) → Storage uploads
6	GenerationDock	POST /api/assist	fetch (Wise enhance) → LLM externo
7	GenerationDock	GET /api/assets	fetch (painel “@”) → tabela assets
8	GenerationDock.handleGenerate	POST /api/generate/video	fetch (submissão)
9	/api/generate/video	lib/supabase/server.createClient	auth + ai_models + profiles + generations + credit_transactions
10	/api/generate/video	lib/plans.planAllows	gating de plano
11	/api/generate/video	lib/piapi/client.buildVideoPayload	adapter por backend
12	/api/generate/video	lib/piapi/client.submitVideoTask → PIAPI POST /task	criação da task
13	StudioFeed	GET /api/generations	fetch (feed) → tabela generations + JOIN ai_models
14	StudioFeed (poll 3 s)	GET /api/generate/status?id=	fetch por geração pendente

#	Origem	Destino	Via
15	/api/generate/status	lib/piapi/client.getTaskStatus → PiAPI GET /task/{id}	status da task
16	/api/generate/status	lib/piapi/client.extractVideoUrl	extração da URL (output_key)
17	/api/generate/status	lib/supabase/service.createServiceClient	download → Storage assets → UPDATE generations → estorno em profiles / credit_transactions
18	Sidebar	useCredits → GET /api/credits	badge de créditos
19	MediaLightbox	useStudioStore (setStartImageUrl , setReferenceTab , setActiveTab)	“Create Video”/“Use as Reference”
20	MediaLightbox	PATCH/DELETE /api/generations/[id]	favoritar/excluir
21	PiAPI (teórico)	POST /api/webhooks/piapi	não registrado nas tasks criadas

4. OBSERVAÇÕES DE RISCO (sem correções nesta etapa)

- Conclusão da geração depende do browser aberto (ausência de fila/worker/webhook ativo).** O único mecanismo que baixa o vídeo e marca `completed` é o polling do `StudioFeed` (3 s). Se o usuário fechar a aba após clicar em Generate, a geração fica presa em `processing` para sempre — sem estorno e sem `result_url` — até que ele reabra o Studio. O webhook `/api/webhooks/piapi` mitigaria isso, mas **não é registrado** no payload das tasks (`config` só tem `service_mode` , `client.ts` l.302).
- Débito/estorno de créditos sem transação atômica.** O débito é feito com `UPDATE profiles SET credits_balance = <valor calculado no app>` (`video/route.ts` l.116–120), padrão read-modify-write sem lock/RPC. Duas gerações simultâneas (ex.: batch de imagens ou duplo clique) podem ler o mesmo saldo e sobrescrever-se mutuamente (race condition), causando débito ou estorno incorreto. O mesmo padrão se repete no estorno (`status/route.ts` l.197–207 e webhook l.50–61).
- Estorno duplicado possível.** O estorno em falha existe em três lugares (`video/route.ts`, `status/route.ts`, webhook). O `status/route.ts` só estorna quando ELE detecta `state === "failed"` —

mas como várias abas/polls simultâneos podem chegar juntos antes do UPDATE, e o webhook (se um dia for ativado) também estorna, não há idempotência (nenhuma checagem de `credit_transactions` já existente com `reason='refund'` para a mesma geração).

4. **Divergência entre migrations e schema de produção.** `supabase/migrations/0001_init.sql` define colunas `modality`, `credits_charged`, `credits`, `generation_id` — mas o código usa `type`, `credits_used`, `credit_cost`, `related_job_id` (conforme `SCHEMA_PRODUCAO.md`). As migrations do repositório **não reproduzem o banco real**; qualquer ambiente novo criado a partir delas quebraria o app.
5. **Duplicação de rotas/fontes de créditos.** `/api/me` e `/api/credits` retornam praticamente o mesmo dado; o dock usa uma e a Sidebar outra (via `useCredits`). Saldo pode divergir visualmente entre dock e sidebar até o refetch de 30 s.
6. **Código morto/órfão:** `src/hooks/use-generation.ts` (polling paralelo nunca usado) e `src/components/studio/media-gallery.tsx` (sem consumidores). Risco de manutenção: alterações futuras podem ser feitas no arquivo errado.
7. **Inconsistência de path no Storage entre status e webhook.** Status route grava `{user}/video/{id}.mp4` (singular); webhook grava `{user}/videos/{id}.mp4` (plural). Se o webhook for ativado, o mesmo vídeo pode ser persistido em dois paths e o `result_url` dependerá de quem chegar por último.
8. **resolution do usuário não é enviada tal-qual à PiAPI.** O dock envia `resolution` no body, e a rota grava em `generations.params`, mas `buildVideoPayload` **deriva a resolução do quality** (ex.: seedance l.312, Wan l.363, veo l.409) e ignora o valor selecionado. O seletor de resolução da UI, portanto, tem efeito apenas indireto/nulo em vários backends — fonte clássica de “o vídeo não saiu na resolução escolhida”.
9. **duration parcialmente ignorada em Kling classic/turbo.** Para `kling classic` e `kling-turbo`, a duração enviada é 5 ou 10 derivada de `quality` (client.ts l.436, 529), não do slider do usuário. UI e payload divergem.
10. **Extração de URL frágil / acoplada a output_key.** Se o modelo no banco tiver `output_key` errado (já ocorreu com Seedance em auditoria anterior), a rota de status devolve `processing` para sempre (l.80–88) — geração completa na PiAPI, mas nunca finaliza no app, sem timeout e sem estorno.
11. **Sem timeout/expiração de gerações.** Não há mecanismo que marque como `failed` gerações presas em `processing` há horas (nem job, nem verificação no polling). Créditos ficam retidos indefinidamente.
12. **Cooldown de alto custo não implementado.** `HIGH_COST_COOLDOWN_SECONDS` / `HIGH_COST_THRESHOLD_CREDITS` existem em `constants.ts` mas nenhum código do fluxo de vídeo os aplica — regra de negócio declarada e não implementada.
13. **Buckets públicos.** `uploads` e `assets` são públicos; URLs de referência e resultados são adivinháveis por quem conhecer o padrão de path (uuid mitiga, mas não há ACL por usuário no acesso de leitura).
14. **Download do vídeo em memória na rota API.** `status/route.ts` faz `fetch(providerUrl) + arrayBuffer()` (l.101–102) dentro de uma rota serverless/Node — vídeos grandes (1080p, 15 s) podem estourar memória/tempo de resposta, e o mesmo download pode ser disparado em paralelo por múltiplos polls simultâneos (sem lock).

Fim do relatório da ETAPA 1. Nenhum arquivo de código foi modificado; nenhum commit foi realizado; nenhuma chamada à PiAPI foi feita durante este mapeamento.